A Systematic Benchmark of Supervised and Unsupervised Graph Embedding Methods

1st Κωνσταντίνα Κουλακτσίδου
dept. Electrical and Computer Engineering
National Technical University of Athens
Athens, Greece
A.M.: 03121800, Team ID: 20
konstantinakoulaktsidou@gmail.com

2nd Φραγκίσκα Κιμίνου
dept. Electrical and Computer Engineering
National Technical University of Athens
Athens, Greece
A.M.: 03121009, Team ID: 20
email

3rd Νικόλαος Μωραΐτης
dept. Electrical and Computer Engineering
National Technical University of Athens
Athens, Greece
A.M.: 03121172, Team ID: 20
nickmoraitis5@gmail.com

***Abstract*---Graph embedding methods provide low dimensional representations of graphs for downstream learning tasks. In this work, we compare unsupervised and supervised graph embedding techniques on real world benchmark datasets. We evaluate Graph2Vec, NetLSD and the Graph Isomorphism Network (GIN) on MUTAG, ENZYMES and IMDB-MULTI using graph classification and clustering, considering accuracy, computational efficiency and scalability. Results show that GIN achieves the highest classification accuracy at a higher computational cost, while Graph2Vec offers a favorable balance between performance and efficiency. NetLSD demonstrates competitive clustering performance with minimal overhead. Overall, each method shows distinct trade offs and maintains reasonable robustness under structural perturbations providing practical guidance for method selection under different data and computational constraints.**

***Index Terms*---Graph2Vec, NetLSD, GIN, Graph Embedding, Graph Classification, Graph Clustering**

I. Introduction

Graphs are widely used to model relationships and interactions in data from multiple domains, such as network analysis, bioinformatics, and chemistry. Many learning tasks are defined at the graph level. However, graphs exhibit irregular structures with variable sizes and features, which pose challenges for traditional machine learning algorithms.

Graph embedding methods address these limitations by mapping entire graphs into fixed-size vector representations, enabling standard machine learning techniques to be applied efficiently to downstream tasks. Early approaches based on handcrafted features or graph kernels suffer from high computational complexity or limited expressiveness [6], [7]. More recent methods automatically learn graph representations and can be broadly categorized into unsupervised [1], [2] and supervised techniques [3].

Figure 1 illustrates the conceptual difference between unsupervised and supervised graph embedding methods. In the supervised setting node representations are learned solely from the graph structure and feature similarity without access to label information. As a result, the embedding space organizes nodes into clusters which may correspond to latent communities but are not explicitly aligned with semantic classes. In contrast, supervised graph embedding methods leverage labeled data during training to shape the embedding space. Nodes belonging to the same class are encouraged to be close as we can see from the figure, while nodes from different classes are pushed apart leading to embeddings that are explicitly discriminative and separable by a decision boundary. This distinction highlights how supervision guides representation learning toward task specific objectives, such as node classification, whereas unsupervised embeddings primarily capture the underlying geometry and connectivity of the graph.

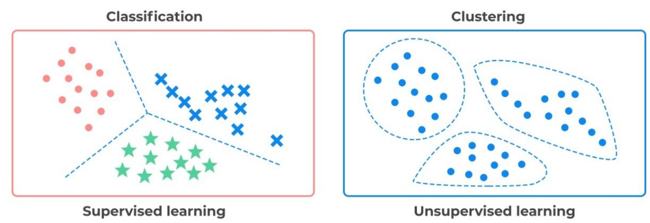

Figure 1: Comparison between unsupervised (a) and supervised learning (b).

Despite their success, supervised methods require labeled data and typically incur higher computational costs due to iterative training, whereas unsupervised methods do not require labels and are generally more efficient. However, systematic comparisons between these two paradigms remain limited, especially on small benchmark datasets where robustness and efficiency are critical [9], [10].

In this work, we focus on the EMB1 setting and our purpose is to compare unsupervised and supervised graph embedding methods on small benchmark datasets from TUDataset. We evaluate Graph2Vec, NetLSD and GIN under a unified experimental framework, considering not only classification performance but also clustering quality, computational cost and stability to structural perturbations. To guide our empirical evaluation, we address the following questions:

- **RQ1:** How do unsupervised graph embedding methods compare to supervised GNN-based approaches in terms of classification performance on small benchmark datasets?
- **RQ2:** How well do different graph embeddings capture

Algorithm 1 Graph2Vec

Require: Graph set $\mathcal{G}$, WL depth D, dim δ, epochs e, lr α
Ensure: Graph embeddings $\Phi \in \mathbb{R}^{|\mathcal{G}| \times \delta}$

Initialize $\Phi \sim \mathcal{U}(-0.5/\delta, 0.5/\delta)$
for $epoch = 1$ to e **do**
  Shuffle $\mathcal{G}$
  for each $G_i \in \mathcal{G}$ **do**
    for $d = 0$ to D **do**
      for each node $n \in G_i$ **do**
        $sg \leftarrow$ WLSubgraph(n, G_i, d)
        Update Φ_i via Skip-Gram(sg, α)
      end for
    end for
  end for
end for
return Φ

Algorithm 2 WLSubgraph

Require: Node n, graph $G = (N, E, \lambda)$, depth d
Ensure: Rooted subgraph label sg

$sg \leftarrow \lambda(n)$
for $i = 1$ to d **do**
  $sg \leftarrow \text{hash}\big(sg \,\|\, \{\lambda(u) : u \in \mathcal{N}(n)\}\big)$
end for
return sg

Algorithm 3 NetLSD (Heat Trace Signature)

Require: Graph $G = (V, E)$, times $T = \{t_k\}_{k=1}^{K}$, Laplacian type $\mathcal{L}$, eigenvalues m
Ensure: Graph signature $s \in \mathbb{R}^K$

Build adjacency A and degree D
$L \leftarrow \begin{cases} I - D^{-1/2} A D^{-1/2}, & \mathcal{L} = \text{norm} \\ D - A, & \mathcal{L} = \text{comb} \end{cases}$
Compute smallest m eigenvalues $\{\lambda_i\}_{i=1}^{m}$ of L
for $k = 1$ to K **do**
  $s_k \leftarrow \sum_{i=1}^{m} e^{-t_k \lambda_i}$
end for
$s \leftarrow \log(s)$ ▷ optional
$s \leftarrow$ Normalize(s) ▷ optional
return s

graph similarity, as reflected by clustering quality?

- **RQ3:** What are the computational efficiency trade-offs between unsupervised and supervised methods?
- **RQ4:** How robust are graph embeddings to structural perturbations of the input graphs?

So our contributions are summarized as follows:

- A unified and fair experimental framework for comparing supervised and unsupervised graph embedding methods on small datasets
- An analysis of performance–efficiency trade offs, and
- An evaluation of embedding robustness under structural perturbations.

II. Graph Embedding Methods

This section describes the graph embedding methods evaluated in this work.

A. *Graph2Vec*

We begin with Graph2Vec [1], an unsupervised whole-graph embedding method that learns fixed-length vector representations in a manner analogous to Doc2Vec for documents. The core idea is to treat each graph as a document and its rooted subgraphs, corresponding to local node neighborhoods, as words. Under this formulation, graphs that share similar structural patterns are expected to be mapped to nearby points in the embedding space.

More specifically, Graph2Vec first applies the Weisfeiler–Lehman (WL) relabeling procedure to extract rooted subgraphs around each node up to a specified depth D (Algorithm 2). These rooted subgraphs are then aggregated across all graphs and WL iterations to form a global vocabulary, referred to as a *bag of subgraphs*. Finally, a skip-gram model is trained to learn graph-level embeddings by maximizing the likelihood of observing the rooted subgraphs associated with each graph (Algorithm 1).

B. *NetLSD*

NetLSD [2] represents a graph using the spectrum of its graph Laplacian, capturing global structural information. The underlying idea is that the eigenvalues of the Laplacian matrix encode important properties of the graph structure. Rather than using the raw eigenvalues directly, NetLSD summarizes the spectral information through a diffusion process, specifically via the heat kernel (Algorithm 3).

A key advantage of NetLSD is its permutation invariance and scalability. To improve computational efficiency, the method approximates the Laplacian spectral distribution using stochastic trace estimation, thereby avoiding full eigendecomposition. As a result, NetLSD maintains stable performance even for large graphs.

C. *GIN*

The Graph Isomorphism Network (GIN) [3] is a supervised graph neural network designed to maximize the discriminative power of GNNs. In GIN, node representations are iteratively updated by aggregating information from neighboring nodes. At each layer, node features are updated using a sum aggregation function followed by a multilayer perceptron (MLP), which enables the model to distinguish different graph structures effectively (Algorithm 4). The resulting node embeddings are then aggregated via a global pooling operation to obtain a graph-level representation. This representation is subsequently passed to a classifier and trained end-to-end using labeled data (Algorithm 5).

In contrast to unsupervised methods, GIN learns task-specific embeddings that are optimized directly for classification performance. While this typically leads to higher accuracy, it requires labeled datasets, which are often difficult to obtain, and incurs higher computational costs due to iterative training.

III. Datasets

The experiments in this work are conducted on three datasets from TUDataset: MUTAG, ENZYMES, and IMDB-MULTI. These datasets are small but diverse, covering domains such as chemistry, biology, and social networks. They

Algorithm 4 GIN Forward Pass

Require: Graph $G = (V, E)$, node features $\{h_v^{(0)}\}$, layers L
Ensure: Graph embedding z_G
for $k = 1$ to L **do**
  for all $v \in V$ **do**
    $h_v^{(k)} \leftarrow \text{MLP}^{(k)}\Big((1 + \epsilon^{(k)}) h_v^{(k-1)} + \sum_{u \in \mathcal{N}(v)} h_u^{(k-1)}\Big)$
  end for
end for
$z_G \leftarrow \text{READOUT}(\{h_v^{(L)}\})$
return z_G

Algorithm 5 Training GIN

Require: Dataset $\mathcal{D}$, epochs E, lr α, batch size B
Ensure: Trained parameters Θ
Initialize Θ
for $epoch = 1$ to E **do**
  for mini-batch $\mathcal{B} \subset \mathcal{D}$ **do**
    for all $(G, y) \in \mathcal{B}$ **do**
      $z_G \leftarrow \text{GINForward}(G)$
      $\hat{y} \leftarrow \text{Classifier}(z_G)$
    end for
    $\mathcal{L} \leftarrow \frac{1}{|\mathcal{B}|} \sum \ell(\hat{y}, y)$
    Update Θ (e.g., Adam)
  end for
end for
return Θ

are standard benchmarks in the graph representation learning literature [1]–[3] and are commonly used to evaluate graph embedding and classification methods.

A. MUTAG

MUTAG [4] is a molecular graph dataset consisting of chemical compounds, where each graph represents a molecule, nodes correspond to atoms, and edges represent chemical bonds. The task is to classify molecules into two classes according to their mutagenic effect on a bacterium. The dataset contains a relatively small number of graphs with discrete node labels representing atom types. Graphs in MUTAG are typically small and sparse, making the dataset suitable for fast experimentation.

B. ENZYMES

The ENZYMES [5] dataset is derived from protein tertiary structures and consists of graphs representing enzymes. Nodes correspond to secondary structure elements, while edges indicate spatial proximity or interactions between residues. Each graph is labeled according to the enzyme's functional class, resulting in a multiclass classification task with six classes.

C. IMDB-MULTI

IMDB-MULTI [5] is a social network dataset constructed from movie collaboration data. Each graph represents the ego network of actors appearing in a movie, where nodes correspond to actors and edges represent co-appearance relationships. Unlike molecular and biological datasets, IMDB-MULTI does not provide node labels or attributes.

IV. Experimental Setup

A. Experimental Pipeline

Our experimental pipeline follows a unified and reproducible procedure across all datasets and methods. For each dataset, input graphs are first processed and converted into fixed-length vector representations using the selected graph embedding methods. These embeddings are then used for three downstream tasks including graph classification, clustering and stability analysis.

For *unsupervised* methods (Graph2Vec and NetLSD), graph representations are learned independently of any label information. The resulting embeddings are computed once per configuration and stored for subsequent evaluation. Classification is performed by training standard machine learning models, as we will refer to them in detail in the next section, on top of the learned graph vectors. For the *supervised method* (GIN), graph representations are learned jointly with the classification objective through end-to-end training. Meaning that graph level embeddings are extracted from the trained network and used both for direct classification performance assessment and for comparative analysis against unsupervised methods.

All experiments are conducted consistently across embedding dimensions of **64**, **128** and **256** in order to study the effect of representation capacity on performance and computational cost.

B. Execution Environment and Hardware Configuration

All experiments are executed on Linux-based systems equipped with multi-core CPUs and sufficient main memory to support large-scale graph embedding computation. We use a combination of **CPU-** and **GPU-based** environments selected according to the computational requirements of each embedding method. Specifically, unsupervised graph embedding methods (Graph2Vec and NetLSD) are executed on CPU-based systems as they don't require accelerated hardware and exhibit modest memory and runtime demands. On the other side, supervised GNN-based models (GIN) are trained using GPU acceleration to efficiently handle all the processing operations. Training and evaluation of GIN models are performed using an at first **NVIDIA T4 GPU** available through Google Colab's free tier and later using an **NVIDIA 3060 GPU**, which provides sufficient computational resources for all datasets considered in this study.

During execution, runtime and memory usage are monitored to capture embedding generation time, training time, and peak memory consumption for computational efficiency analysis. This hybrid execution strategy enables efficient experimentation while maintaining comparable conditions across all the three methods and three datasets.

C. Data Splits and Reproducibility

For each dataset and embedding dimensionality, embeddings are split into 80% training and 20% test sets. To ensure fair comparison across methods, the same data splits are used for all embedding approaches within each experimental configuration.

Random seeds are fixed across all experiments, including dataset spliting, embedding generation, classifier training etc., to reduce variance from stochastic effects and to ensure that observed performance differences can be attributed to the embedding methods themselves. All reported results are obtained using identical splits, configurations, and evaluation scripts.

D. Implementation and Software

All experiments are implemented in Python. Unsupervised graph embeddings are generated using publicly available implementations of Graph2Vec and NetLSD integrated through the `KarateClub` library. Graph neural network models are implemented using `PyTorch` and `PyTorch Geometric`, which provide efficient primitives for graph construction, batching and message passing. Graph datasets are loaded and preprocessed using the `TUDataset` interface from PyTorch Geometric to ensure consistent handling of graph structures and labels. For datasets without explicit node attributes we use and construct default structural features in order to enable GNN training. The resulting embeddings are stored in CSV format, with one row per graph containing the graph label and the corresponding embedding vector. Downstream classification on unsupervised embeddings is performed using `scikit-learn`. Prior to training, all embeddings are standardized using `StandardScaler`.

The following classifiers are employed: Support Vector Machines with RBF kernel, Multilayer Perceptrons, Random Forests, Logistic Regression, Gradient Boosting and a Voting Classifier combining heterogeneous models. Classifier selection and configuration are kept consistent across methods to isolate the effect of the embedding quality.

The supervised GIN model follows the architecture and training protocol described in Algorithms 4 and 5. GIN models consist of stacked GINConv layers parameterized by multilayer perceptrons, followed by batch normalization, ReLU activation, dropout, and residual connections. Graph-level representations are obtained using additive global pooling. Jumping Knowledge with concatenation is employed to aggregate information from multiple layers.

Several training options are enabled to improve stability and generalization, including feature normalization, graph-level feature augmentation and label smoothing. Early stopping is applied during training to prevent overfitting particularly on small datasets.

E. Hyperparameter Configuration

Embedding configuration dimensions are set to the values we already mention. Classifier hyperparameters are adapted to dataset size to mitigate overfitting on small benchmarks ensure stable training. For GIN, multiple training configurations are explored to analyze the effect of model capacity and regularization. Training is performed using learning rates of 1×10^{-3} and 5×10^{-4}. Dropout rates of 0.0 and 0.4 are evaluated to assess the impact of regularization. The number of GIN layers is selected per dataset to reflect differences in graph size and structures, with shallower architectures used for social network graphs and deeper architectures for molecular and biological graphs. Specifically, after many experiments the number of layers is set to 3 for IMDB-MULTI, 5 for MUTAG, and 6 for ENZYMES. Batch sizes are set to 64 for MUTAG and IMDB-MULTI, and 32 for ENZYMES after.

In addition to architectural variations, several training options and design choices are enabled to improve stability and generalization. Graph-level feature augmentation is applied by concatenating simple structural statistics, including the number of nodes, number of edges and graph density, to the learned graph embeddings. Additive global pooling is employed to aggregate node representations into graph-level embeddings, as it has been shown to provide stable performance across datasets. Feature normalization is applied after node feature construction to improve optimization behavior. Furthermore, label smoothing with a factor of 0.05 is applied to the cross-entropy loss in order to reduce overconfidence during training.

Training is conducted for up to 200, 300, 500, or 800 epochs depending on the dataset, with early stopping enabled to mitigate overfitting and reduce sensitivity to the chosen maximum number of epochs. In practice we observed that these upper bounds are conservative, as arly stopping consistently triggers well before convergence reaches the maximum epoch limit, typically within the first 200 epochs across all datasets.

This hyperparameter exploration allows us to analyze how architectural depth, embedding dimensionality and regularization affect both accuracy and the quality of the learned graph embeddings.

V. Evaluation Methodology

A. Graph Classification Evaluation

We evaluate graph embeddings on three benchmark datasets from TUDataset. Unsupervised methods are evaluated under a common downstream classification pipeline, while the supervised model follows its native end-to-end training protocol.

Classification performance is assessed using Accuracy, F1-score, and Area Under the ROC Curve (AUC). In addition to predictive performance, we report computational metrics including training time, embedding generation time, and memory consumption, measured via peak memory usage and resident set size (RSS). To study the effect of representation capacity, all methods are evaluated using three different embedding dimensions. Smaller embedding dimensions were initially considered to reduce computational cost and memory consumption. However, evaluating higher-dimensional representations allows us to examine the trade-off between efficiency and representational capacity, as discussed in the results section.

B. Evaluation of Unsupervised Embeddings

For unsupervised methods, graph embeddings are evaluated by training standard classifiers on top of the learned representations.

Prior to classification, embeddings are standardized using a z-score normalization (sklearn's StandardScaler) to ensure consistent feature scaling, which is particularly important for distance- and margin-based classifiers. The following classifiers are used for evaluation:

- Support Vector Machine (SVM) with an RBF kernel and probability calibration enabled for AUC computation,
- Multilayer Perceptron (MLP), with hidden layer sizes adapted to the embedding dimensionality, using deeper architectures for higher-dimensional embeddings,
- Random Forest classifier, with the number of trees and maximum depth adjusted according to dataset size to mitigate overfitting on small benchmarks,
- Logistic Regression with a multinomial formulation for multiclass datasets, and
- Gradient Boosting classifier with moderate tree depth.

In addition, an ensemble Voting Classifier combining Random Forest, Gradient Boosting and SVM is employed to assess the benefits of combining heterogeneous decision boundaries.

C. Evaluation of Supervised Graph Embeddings

For supervised evaluation, GIN produces graph-level representations through global pooling of node embeddings and is trained end-to-end using labeled data. The resulting graph representations are evaluated directly through the model's classification output, without training additional downstream classifiers.

D. Clustering Evaluation

To assess how well graph embeddings preserve graph-to-graph similarity, we evaluate clustering performance on the learned representations. Clustering quality reflects the ability of an embedding to capture global structural properties in the absence of label supervision.

We applied both K-Means and Spectral clustering accross all dataset-embedding combinations in order to observe how embedding methods and embedding dimensions impact performance. We evaluated the clustering quality using Normalized Mutual Information (NMI), Adjusted Rand Index (ARI), Accuracy and Silhouette scores. We visualized the clustering results using scatter plots and provided a holistic overview of the results using line plots that showed the performance of each algorithm on every dataset with respect to the embedding dimension. We need to note that while Graph2Vec and NetLSD utilized standard dimensions, the GIN embeddings employed dataset-dependent dimensions.

E. Stability Analysis

Real world graph data are often noisy, with missing edges or corrupted node attributes. To assess the robustness of graph embeddings under such perturbations, we perform a stability analysis by introducing controlled modifications to graph structures and node features.

For each dataset, perturbed versions of the graphs are generated using two types of perturbations. First, edge perturbation is applied by randomly removing 10% of existing edges and adding an equal number of randomly generated edges. Second,

Dataset	Dim	Epochs	Acc	F1	AUC
MUTAG	64	200	0.842	**0.825**	**0.885**
	128	200	0.842	0.808	**0.885**
	256	200	**0.895**	**0.864**	0.859
ENZYMES	64	300	0.433	0.428	0.703
	128	300	0.483	0.492	0.744
	256	300	**0.617**	**0.615**	**0.809**
IMDB-MULTI	64	200	0.500	**0.496**	0.686
	128	200	0.500	0.486	0.678
	256	200	**0.507**	0.478	**0.714**

Table I: GIN classification best performance. With epochs $\in \{200, 300\}$

node attribute perturbation is introduced by randomly shuffling node features between pairs of nodes within each graph.

After perturbation, graph embeddings are recomputed using Graph2Vec, NetLSD, and GIN with the same scripts and configurations as in the original experiments. This ensures that any observed differences in the embeddings are attributable solely to the introduced perturbations.

Embedding stability is quantified using two complementary similarity measures. Cosine similarity is used to assess directional consistency between the original and perturbed embeddings,

$$\cos(z_i, z_i') = \frac{z_i^\top z_i'}{\|z_i\|_2 \|z_i'\|_2},$$

while the Euclidean distance measures absolute deviation,

$$\|z_i - z_i'\|_2.$$

For each dataset, method and experimental configuration, we report the mean, median and standard deviation of both cosine similarity and Euclidean distance across all graphs. To evaluate the practical impact of perturbations, we additionally repeat the graph classification and clustering experiments on the perturbed datasets using the same evaluation protocols.

VI. Results

A. Before Permutation

For the graph classification tasks before any perturbations, we present the performance of each embedding method across the three TUDatasets with accuracy, F1-score and AUC metrics for each method in Tables I, II and III. We preferered to show only the best test configuration per dataset x dim results and for the epochs 200 and 300 (for GIN) for the reason we mentioned in earlier section.

For the MUTAG, we can see that GIN excels in classification accuracy, but NetLSD provides superior probabilistic separation as indicated by its higher AUC scores. AUC slightly decreases with increasing dimensions for GIN, showing that higher classification accuray does not always mean that the method has confidence in its predictions. So, our conclusion is that spectral info captured by NetLSD is beneficial for this dataset. For ENZYMES, the gap between GIN and the other two methods is clearer in all metrics. GIN clearly dominates here and AUC metric confirms that GIN is more confident in its predictions. For IMDB-MULTI, when node features

Dataset	Dim	Model	Acc	F1	AUC
MUTAG	64	RF	0.711	0.656	0.775
	128	RF	**0.763**	**0.754**	**0.797**
	256	RF	0.737	0.731	0.745
ENZYMES	64	Ens	0.258	0.252	0.571
	128	RF	0.225	0.219	**0.600**
	256	RF	**0.317**	**0.310**	0.591
IMDB-MULTI	64	RF	**0.437**	**0.435**	**0.601**
	128	GB	0.427	0.426	0.583
	256	SVM	0.433	0.426	0.599

Table II: Best Graph2Vec classification results. For each dataset dimension pair, the classifier with the highest test accuracy is reported.

Dataset	Dim	Model	Acc	F1	AUC
MUTAG	64	Ens	**0.895**	**0.892**	0.908
	128	Ens	0.868	0.867	0.883
	256	LR	0.868	0.867	**0.929**
ENZYMES	64	RF	0.300	0.302	0.666
	128	RF	0.308	0.312	0.653
	256	RF	**0.325**	**0.324**	**0.673**
IMDB-MULTI	64	SVM	**0.480**	**0.477**	0.625
	128	RF	0.463	0.465	0.633
	256	RF	0.457	0.456	**0.648**

Table III: Best NetLSD classification results. For each dataset dimension pair, the classifier with the highest test accuracy is reported.

are absent, GIN and NetLSD have similar behavior, with first method being slightly ahead.

So overall, the supervised method is the most effective method and NetLSD is second best, while Graph2Vec shows the weakest performance across all datasets and metrics. The results confirm that supervised GNNs provide the highest predictive power, while spectral descriptors such as NetLSD remain surprisingly strong for unsupervised graph representation learning.

The experimental results reveal clear trade offs between classification accuracy and computational efficiency across GIN, Graph2Vec and NetLSD, as illustrated in Figures 2-6. All figures are computed with the average results across the three dimensions and three datasets. We chose these 3 images, from the 39 we generated for the classification section, to summarize and make our conclusions.

Figure 2 presents a heatmap of average accuracy across

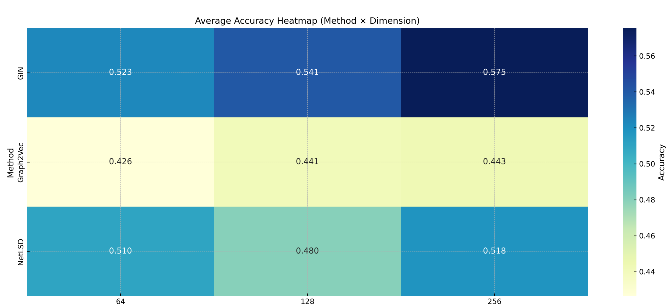

Figure 2: Average accuracy heatmap across methods and embedding dimensions.

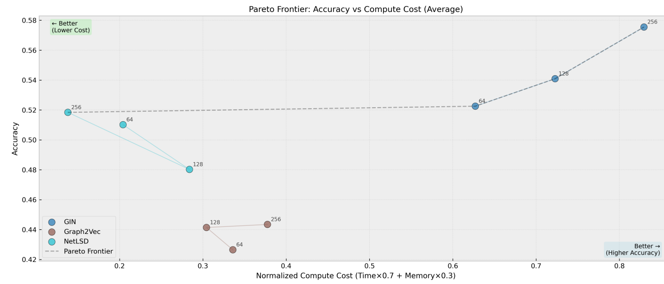

Figure 3: Pareto frontier illustrating the trade-off between accuracy and normalized compute cost.

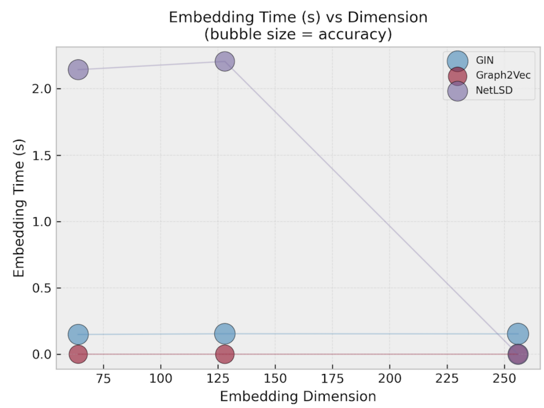

Figure 4: Embedding time as a function of embedding dimension.

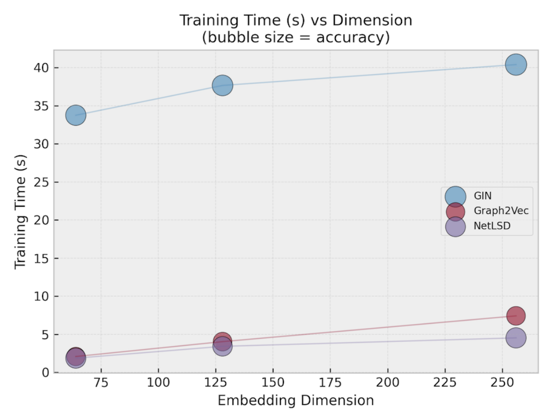

Figure 5: Training time as a function of embedding dimension.

methods and embedding dimensions. We can confirm that GIN consistently outperforms the other two methods across all dimensions.

The Pareto frontier in Figure 3 highlights the balance between efficiency and performance. NetLSD lies on the left most side of the optimal region, meaning that it achieves reasonable accuracy with minimal compute cost. GIN dominates with its accuracy, but with a significantlt higher compute cost. Graph2Vec is at the lower part of the graph, showing both lower accuracy from both other methods and higher

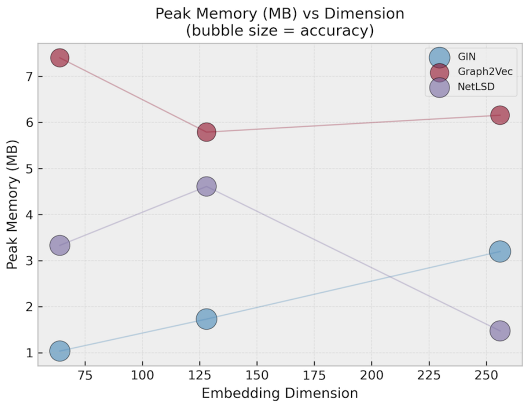

Figure 6: Peak memory usage as a function of embedding dimension.

compute cost than NetLSD. From the same figure, we can see that GIN benefits monotonically from increased embedding dimensionality, achieving the highest average accuracy at 256 dimensions. In contrast, Graph2Vec and NetLSD reach their peak accuracy at 128 dimensions, beyond which performance declines, indicating diminishing returns with higher dimensions.

Figures 4-6 break down the compute cost into embedding time, training time and peak memory usage. Training time (Figure 5) is dominated by GIN and increases substantially with embedding dimension. Embedding time (Figure 4) is highest for NetLSD at lower dimensions due to its spectral computations but decreases at higher dim sizes. Peak memory usage (Figure 6) is highest for Graph2Vec, while NetLSD remains more memory efficient.

Continuing with the clustering task, tables IV, V and VI are a compact version from the original clustering results tables. Best metrics are chosen based on highest ACC and silhouette as a tiebreaker. Graph2Vec (Table IV) shows the weakest clustering performance. Something we expected based on its classification results and its confidence in seperations (as seen from AUC scores). With the low ACC scores and zero or negative ARI scores, we can say that Graph2Vec has poor aligment with the true class labels. NetLSD has the strongest clustering performance among the three methods. For MUTAG, it achieves high ACC and solid NMI and ARI scores indicating good alignment with true labels. For ENZYMES and IMDB-MULTI, it consistently outperforms Graph2Vec, though the overall clustering quality is moderate. GIN creates mixed feelings here. The fact that is a supervised representation makes it not optimal for clustering tasks, meaning for unsupervised tasks. GIN performs better than Graph2Vec, but worse than NetLSD.

The experimental results for clustering evaluation are well supported by the theoretical facts. More specifically, the graph2vec method emphasizes on local rooted substructures treating graphs as unorderdered colections of neighborhood patterns. This method is effective for capturing local similarities while it lacks a mechanism for representing global structures. On the contrary NetLSD embeds graphs using spectral summaries of the Laplacian, which are permutation invariant and capture global diffusion behaviour. Such characteristics are well suited for clustering. This exlplains why NetLSD consitently forms semantically coherent clusters despite being label-free. GIN embeddings are designed to maximise separablity rather than maintaining inter-graph distances. Consequently embeddings using GIN perform poorly during clustering. The theoretic differences are clearly represented in figures 7 to 12

Data	Dim	Alg	NMI	ARI	ACC	Sil
MUTAG	64	spec	**0.048**	**-0.019**	**0.532**	0.285
	128	km	0.033	**-0.019**	0.521	0.305
	256	spec	0.037	-0.027	0.511	**0.315**
ENZ	64	km	0.027	0.006	**0.223**	0.205
	128	km	**0.030**	**0.010**	0.220	**0.283**
	256	spec	0.021	0.005	0.220	0.210
IMDB	64	km	**0.013**	**0.011**	**0.402**	0.506
	128	km	0.011	0.010	0.399	0.552
	256	km	0.011	0.010	0.400	**0.584**

Table IV: Best Graph2Vec clustering results (highest ACC; ties by Silhouette).

Data	Dim	Alg	NMI	ARI	ACC	Sil
MUTAG	64	km	**0.247**	**0.302**	**0.777**	**0.576**
	128	km	**0.247**	**0.302**	**0.777**	**0.576**
	256	km	**0.247**	**0.302**	**0.777**	**0.576**
ENZ	64	spec	**0.038**	**0.014**	**0.250**	**0.168**
	128	spec	**0.038**	**0.014**	**0.250**	**0.168**
	256	spec	**0.038**	**0.014**	**0.250**	**0.168**
IMDB	64	km	**0.021**	**0.016**	**0.410**	**0.546**
	128	km	**0.021**	**0.016**	**0.410**	**0.546**
	256	km	**0.021**	**0.016**	**0.410**	**0.546**

Table V: Best NetLSD clustering results (highest ACC).

B. After Permutation

After the permutations where we explained in the section *V*, we present the stability results from the embeddings for each method in Tables VII, VIII IX and X.

The stability behavior of GIN is strongly influenced by training configuration and dataset characteristics. When we optimize for max cosine (closer to value 1), we take models with dropout 0.0 and we have high cosine similarity values (0.66-0.91 range) with gradually decreasing cosine similarity

Data	Dim	Alg	NMI	ARI	ACC	Sil
MUTAG	64	km	0.244	**0.236**	**0.745**	0.445
	128	km	0.209	0.176	0.713	0.406
	256	km	**0.298**	0.135	0.691	**0.621**
ENZ	64	spec	0.082	0.028	0.282	0.029
	128	spec	0.088	0.014	0.288	0.050
	256	spec	**0.221**	**0.128**	**0.365**	**0.076**
IMDB	64	spec	**0.014**	**0.008**	0.378	-0.535
	128	spec	0.011	0.004	0.379	-0.308
	256	spec	0.010	0.006	**0.391**	**-0.245**

Table VI: Best GIN clustering results (highest ACC).

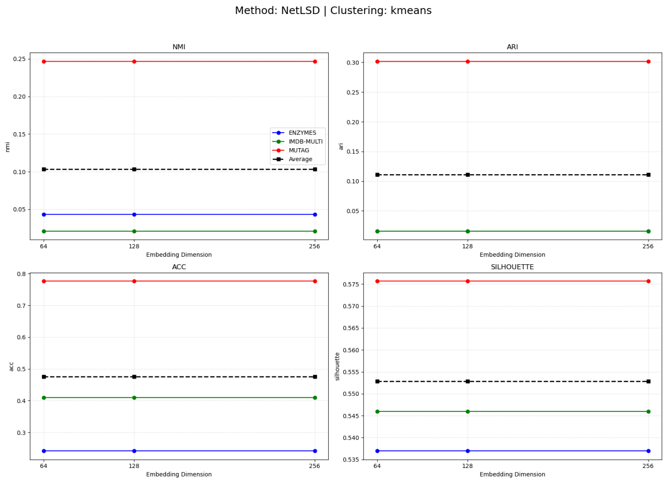

Figure 7: **NetLSD K-Means Performance.** It outperforms others in stability.

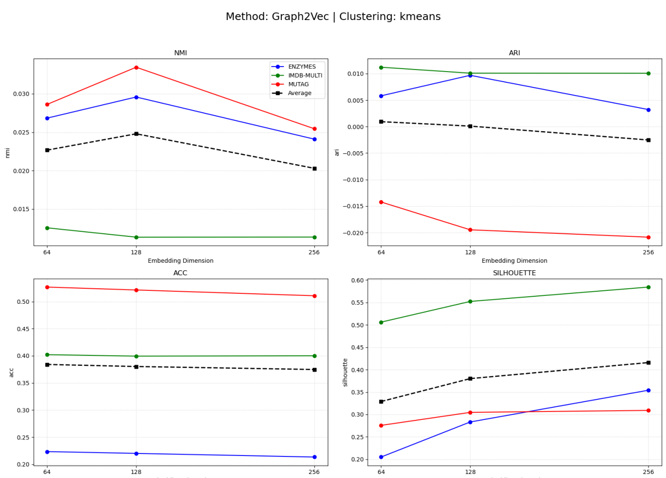

Figure 8: **Graph2Vec K-Means Performance.**

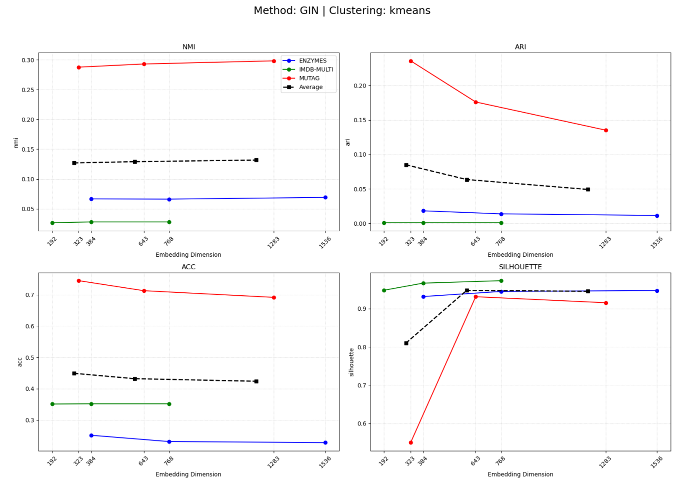

Figure 9: **GIN K-Means Performance.**

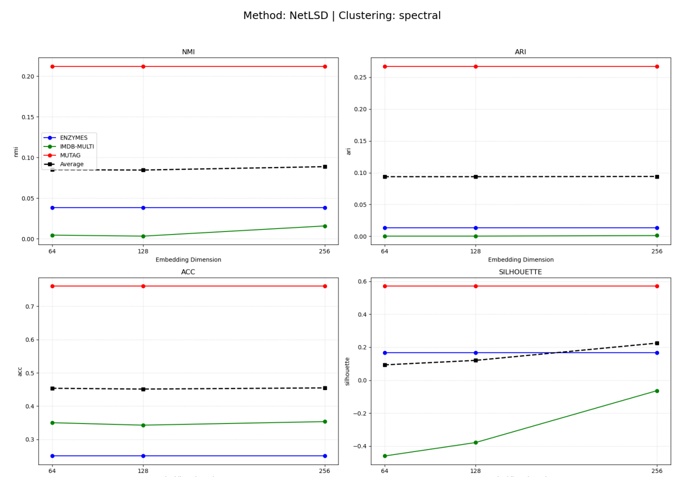

Figure 10: **NetLSD Spectral Clustering.** The trend mirrors K-Means.

as dimensionality increases. However, the corresponding L2 distances are relatively large (ranging from 100 to 1800), meaning significant magnitude changes in embeddings despite relatively preserved directions.

When we optimize for min L2 (closer to value 0), we take models with dropout 0.4 and we have lower cosine similarity values (0.12-0.77 range), indicating more directional changes. However, the L2 distances are much smaller (ranging from 66 to 483), showing reduced magnitude changes. This inverse relationship between cosine similarity and L2

Data	D	LR	Cos	σ	L2	σ
	64	1e-3	**0.918**	**0.027**	**256**	**131**
MUTAG	128	1e-3	0.862	0.032	454	197
	256	1e-3	0.869	0.032	783	381
	64	5e-4	**0.817**	**0.041**	1312	1965
ENZ	128	5e-4	0.783	0.040	**1100**	**813**
	256	5e-4	0.796	0.038	1804	1743
	64	5e-4	0.664	0.070	**117**	**160**
IMDB	128	5e-4	**0.671**	**0.072**	138	259
	256	1e-3	0.666	0.066	182	363

Table VII: GIN embedding stability (best per dataset dim by highest cosine mean, dropout = 0.0)

Data	D	LR	L2	σ	Cos	σ
	64	5e-4	**168**	**43**	0.775	0.016
MUTAG	128	1e-3	263	77	**0.781**	**0.030**
	256	5e-4	455	167	0.752	0.028
	64	5e-4	**402**	**457**	**0.563**	**0.056**
ENZ	128	1e-3	483	1835	0.297	0.085
	256	1e-3	460	1579	0.126	0.080
	64	1e-3	69	118	**0.663**	**0.066**
IMDB	128	1e-3	105	182	0.625	0.068
	256	1e-3	**66**	**132**	0.126	0.072

Table VIII: GIN embedding stability (best per dataset--dim by lowest L2 mean, dropout = 0.4).

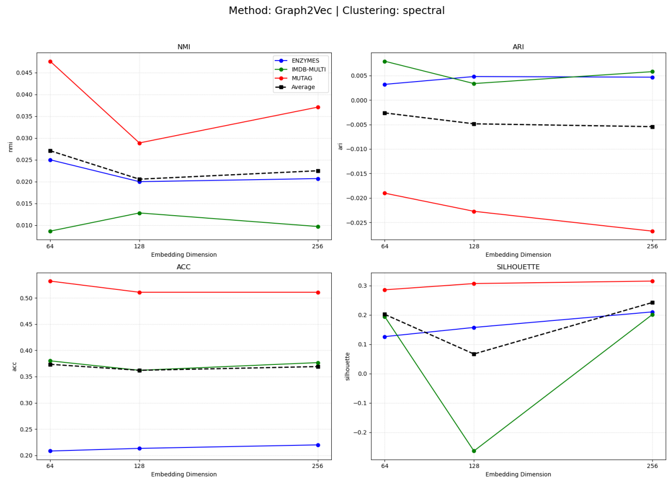

Figure 11: **Graph2Vec Spectral Clustering.**

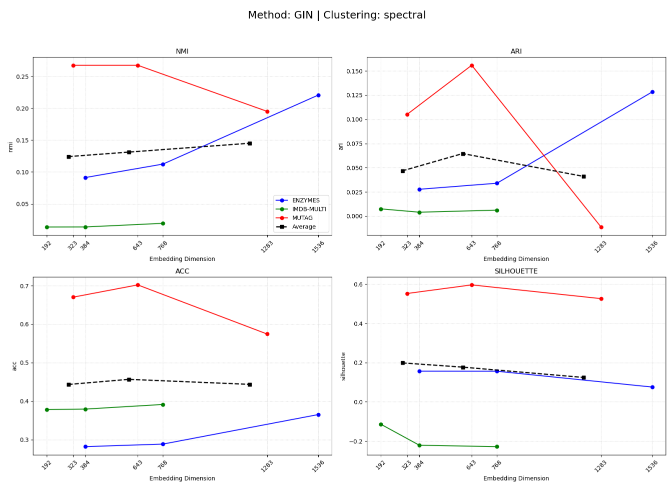

Figure 12: **GIN Spectral Clustering.** Consistently mirrors structural properties.

distance across configurations highlights the trade off between preserving direction versus magnitude in GIN embeddings under perturbations.

Graph2Vec shows strong stability across all datasets and dimensions. Cosine similarity has closer to 1 values and l2 distances are very small. This indicates that Graph2Vec

Data	D	Cos	σ	L2	σ
	64	0.899	0.018	0.133	0.052
MUTAG	128	**0.910**	**0.023**	0.095	0.039
	256	0.899	0.027	**0.068**	**0.027**
	64	0.825	0.029	0.603	0.138
ENZ	128	0.863	0.022	0.545	0.124
	256	**0.896**	**0.020**	**0.487**	**0.117**
	64	**0.719**	**0.062**	**0.672**	**0.325**
IMDB	128	0.696	0.046	0.680	0.317
	256	0.661	0.025	0.691	0.323

Table IX: Graph2Vec embedding stability under perturbations.

Data	D	Cos	σ	L2	σ
	64	-0.818	0.366	**52.3**	**30.2**
MUTAG	128	-0.818	0.366	**52.3**	**30.2**
	256	**0.875**	**0.029**	216	58.7
	64	0.942	0.169	**26.8**	**30.6**
ENZ	128	0.942	0.169	**26.8**	**30.6**
	256	**0.964**	**0.013**	104	66.4
	64	0.658	0.566	**58.8**	**61.6**
IMDB	128	0.658	0.566	**58.8**	**61.6**
	256	**0.962**	**0.027**	127	83.9

Table X: NetLSD embedding stability under perturbations.

Dataset	Dim	Epochs	Acc	F1	AUC
	64	200	0.737	0.562	0.808
MUTAG	128	200	**0.789**	**0.729**	0.615
	256	200	0.737	0.562	**0.885**
	64	300	0.267	**0.219**	0.565
ENZYMES	128	300	0.217	0.187	0.562
	256	300	**0.283**	**0.222**	**0.616**
	64	200	0.420	0.380	**0.611**
IMDB-MULTI	128	200	0.380	0.343	0.596
	256	200	**0.433**	**0.386**	0.598

Table XI: GIN classification best performance. After the perturbations.

embeddings are robust to structural and feature perturbations, maintaining both direction and magnitude. Finally, NetLSD shows mixed stability results. We observe that with dim sizes 256, we have high cosine similarity values and in general small L2 distances (if we compare with GIN). However, at lower dimensions (64 and 128), cosine similarity drops significantly (except for ENZYMES), even becoming negative for MUTAG. L2 distances increase with dimensionality but remain moderate relative to GIN.

So, across the three methods, Graph2Vec is the king of stability, NetLSD achieves very strong stability only at higher dim sizes and GIN shows the most sensitivity to perturbations.

Executing again the classification task after perturbing the graphs we receive the values we see on Tables XI-XIII

References

[1] A. Narayanan, M. Chandramohan, L. Chen, Y. Liu, and S. Saminathan, "graph2vec: Learning Distributed Representations of Graphs," *arXiv preprint arXiv:1707.05005*, 2017.

[2] A. Tsitsulin, D. Mottin, P. Karras, A. Bronstein, and E. Müller, "NetLSD: Hearing the Shape of a Graph," in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '18)*, 2018.

Dataset	Dim	Model	Acc	F1	AUC
	64	RF	0.711	0.700	0.855
MUTAG	128	RF	**0.763**	**0.768**	**0.880**
	256	RF	0.737	0.741	0.877
	64	Ens	0.183	0.174	0.497
ENZYMES	128	RF	**0.25**	**0.251**	**0.581**
	256	RF	0.175	0.162	0.567
	64	RF	**0.38**	**0.378**	**0.551**
IMDB-MULTI	128	GB	0.333	0.334	0.514
	256	SVM	0.370	0.369	0.523

Table XII: Best Graph2Vec classification results. After the perturbations.

Dataset	Dim	Model	Acc	F1	AUC
MUTAG	64	Ens	**0.869**	**0.867**	**0.941**
	128	Ens	0.842	0.842	0.920
	256	LR	0.789	0.785	0.871
ENZYMES	64	RF	0.217	0.211	0.586
	128	RF	0.225	0.221	0.563
	256	RF	**0.233**	**0.228**	**0.605**
IMDB-MULTI	64	SVM	0.357	0.354	0.535
	128	RF	**0.380**	**0.376**	**0.557**
	256	RF	0.353	0.353	0.550

Table XIII: Best NetLSD classification results. After the perturbations.

[3] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How Powerful are Graph Neural Networks?" in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2019.

[4] A. K. Debnath, R. L. López de Compadre, G. Debnath, A. J. Shusterman, and C. Hansch, "Structure–activity relationship of mutagenic aromatic and heteroaromatic nitro compounds. Correlation with molecular orbital energies and hydrophobicity," *Journal of Medicinal Chemistry*, vol. 34, no. 2, pp. 786–797, 1991, doi: 10.1021/jm00106a046.

[5] R. A. Rossi and N. K. Ahmed, "The Network Data Repository with Interactive Graph Analytics and Visualization," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2015.

[6] N. Shervashidze, P. Schweitzer, E. J. van Leeuwen, K. Mehlhorn, and K. M. Borgwardt, "Weisfeiler–Lehman Graph Kernels," *Journal of Machine Learning Research*, vol. 12, pp. 2539–2561, 2011.

[7] S. V. N. Vishwanathan, N. N. Schraudolph, R. Kondor, and K. M. Borgwardt, "Graph Kernels," *Journal of Machine Learning Research*, vol. 11, pp. 1201–1242, 2010.

[8] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The Graph Neural Network Model," *IEEE Transactions on Neural Networks*, vol. 20, no. 1, pp. 61–80, 2009.

[9] F. Errica, M. Podda, D. Bacciu, and A. Micheli, "A Fair Comparison of Graph Neural Networks for Graph Classification," in *Proceedings of the ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[10] V. P. Dwivedi, C. K. Joshi, T. Laurent, Y. Bengio, and X. Bresson, "Benchmarking Graph Neural Networks," *arXiv preprint arXiv:2003.00982*, 2020.

[11] B. Rozemberczki, O. Kiss, and R. Sarkar, "Karate Club: An API Oriented Open-Source Python Framework for Unsupervised Learning on Graphs," in *Proceedings of the 29th ACM International Conference on Information and Knowledge Management (CIKM)*, 2020.

[12] M. Fey and J. E. Lenssen, "Fast Graph Representation Learning with PyTorch Geometric," in *Proceedings of the ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.